Weather Forecasting: An End-to-End Pipeline via Snowflake ML and Apache Airflow

Team 4: Xu Wang, Elina Yin
Department of Data Science
San Jose State University
San Jose, USA
Email: xu.wang@sjsu.edu, elina.yin@sjsu.edu

***Abstract*—This paper presents the implementation of a scalable containerized weather forecasting system. By integrating Apache Airflow and Snowflake for machine learning, the project automates the transition from raw data ingestion to predictive analytics.**

***Index Terms*—Data Engineering, ETL, Machine Learning, Snowflake, Apache Airflow, Docker.**

I. Team Introduction

This project was collaboratively developed by **Lab Team 4** from Data 226 Course. The team utilized an agile approach to integrate machine learning with robust ETL orchestration.

- **Xu Wang (Lead Architect)**: Responsible for the overall system architecture, including the integration of *Snowflake Cortex ML* functions, configuration of *Airflow Variables* for environment portability, and centralized *Git repository management* and documentation.
- **Elina Yin (Data Analyst)**: Focused on the end-to-end data lifecycle, including *Open-Meteo API* data validation, the development of complex *SQL transformation logic* within Snowflake, and *final report wording* .

II. Problem Statement

A. System Objective

Our team aimed to develop an **automated End-to-end weather prediction and analysis system**. The application is designed to autonomously ingest high-resolution meteorological data and provide actionable temperature predictions for specific geographic coordinates (e.g., San Jose and Cupertino). In an era of increasing climate volatility, traditional manual data collection and siloed forecasting methods are insufficient. This system provides a unified platform that bridges the gap between raw data harvesting and predictive business intelligence.

B. The Necessity of Databases and Data Pipelines

The integration of a centralized database and automated data pipelines is critical for the following reasons:

1) **Data Consistency and Integrity**: Weather data from APIs like Open-Meteo is transient and highly granular. A robust cloud database (Snowflake) acts as a *Single Source of Truth*, allowing for historical trend analysis that is impossible with raw API calls alone.
2) **Automation of the ML Lifecycle**: Manual data preparation is the primary bottleneck in machine learning. By utilizing Apache Airflow, we automate the extraction, transformation, and loading (ETL) tasks, ensuring the Snowflake Cortex ML models always operate on the most recent and validated datasets.
3) **Computational Efficiency**: The pipeline architecture enables *In-place AI* via Snowflake Cortex. Instead of moving massive datasets to external environments for processing, the system leverages the database's native compute power to execute forecasting models directly, minimizing latency and security risks.

III. Dataset(s)

The performance and accuracy of the predictive model rely on high-quality historical meteorological records. For this project, we integrated two distinct data streams to facilitate both model training and validation.

A. Data Ingestion and Scope

The primary data source is the **Open-Meteo Historical Weather API**. The pipeline is configured to ingest the most recent **60 days** of historical records to provide a robust baseline for time-series forecasting. The data scope is focused on two key geographic locations in Silicon Valley:

- **San Jose, CA**: (Latitude: 37.33, Longitude: -121.88)
- **Cupertino, CA**: (Latitude: 37.32, Longitude: -122.03)

B. Feature Engineering

To capture the complexity of local weather patterns, we selected three critical meteorological features for each location:

1) *temperature_2m_max*: Daily maximum temperature at 2 meters above ground, used to predict daytime peaks.
2) *temperature_2m_min*: Daily minimum temperature, essential for tracking overnight cooling trends.
3) *precipitation_sum*: Total daily liquid equivalent precipitation, incorporated to identify potential correlations between moisture levels and temperature fluctuations.

C. Target Output

Using the 60-day historical window, the system generates a **7-day forward-looking forecast**. The resulting dataset is a unified time-series structure that aligns historical "Actual"

values with Cortex-generated "Predicted" values, enabling seamless comparative analysis.

IV. System Diagram

The architecture of the proposed weather forecasting system is built upon a modular, containerized framework that ensures scalability and environment consistency. The high-level data flow and component interactions are illustrated in Fig. 1.

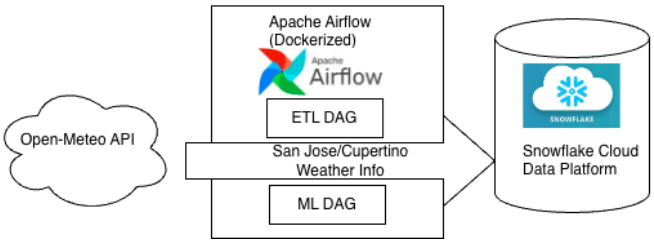

Fig. 1: System Architecture

A. Architectural Components

The system is bifurcated into two primary environments:

1) **Orchestration Layer (Local/Docker)**: Utilizing *Docker Compose*, we deployed a multi-container environment running *Apache Airflow*. This layer manages task scheduling, retry logic, and secure credential handling via Airflow Connections.
2) **Data and ML Layer (Cloud/Snowflake)**: All persistent storage and heavy-duty computations are offloaded to *Snowflake*. This includes raw data ingestion (ETL), SQL-based transformations, and the execution of the *Cortex ML* forecasting engine.

B. Data Pipeline Workflow

The end-to-end workflow follows a rigorous sequence: *API Extraction* → *Staging in Snowflake* → *Cortex Training* → *Forecast Generation*. By decoupling the ingestion logic (Python) from the analytical logic (SQL/ML), the system achieves high fault tolerance and maintainability.

C. Functional Component Analysis

The proposed application system functions as a cohesive unit by orchestrating the flow of data through three primary functional domains. The following analysis discusses how these components collectively solve the problem through their specific database and pipeline interactions.

1) Data Ingestion and Normalization (Airflow ETL): This component serves as the system's raw data entry point. The **data pipeline interaction** is managed by the `Lab1_ETL` DAG, which utilizes a `SnowflakeHook` to securely fetch 60-day historical weather data from the Open-Meteo API. The pipeline processes the raw response into a structured format using the `pd.DataFrame()` library. Subsequently, the **database interaction** involves executing SQL `INSERT` operations to load the structured records into the `WEATHER_HISTORICAL` table, enforcing schema integrity and ensuring data is structurally aligned before the transaction is committed.

2) Predictive Intelligence (Snowflake Cortex ML): This component leverages in-database machine learning to transform static records into forward-looking insights. The **data pipeline interaction** is orchestrated by the `Lab1_ML_Forecasting` DAG, which injects environment-specific Airflow Variables (such as model identifiers and table names) into the SQL execution context to trigger the training sequence. The **database interaction** occurs entirely within the Snowflake environment, where the `SNOWFLAKE.ML.FORECAST` procedure reads directly from the historical tables to generate predictions. This eliminates data egress and ensures that inference results are immediately available for relational joins.

3) Transactional Consolidation (Reporting Layer): The reporting layer resolves data fragmentation by unifying ground-truth observations with predictive inferences. The **data pipeline interaction** coordinates a multi-step SQL script that captures transient inference results via `RESULT_SCAN` and monitors the task's state within a `try...except` block to ensure synchronization. This triggers a final **database interaction** governed by an ACID-compliant transaction (`BEGIN...COMMIT`), which performs a `UNION ALL` operation to merge historical and forecast data into the `WEATHER_FINAL_REPORT` table, providing a single, consistent source of truth for the end-user.

V. Data Schema and Tables

The relational database architecture in Snowflake is designed to support a structured flow from raw ingestion to predictive output. We implemented three distinct relational structures within the `USER_DB_FOX.RAW` schema.

A. Data Dictionary

The system implements a multi-dimensional schema to capture a holistic view of meteorological conditions. The transformation logic in Python normalizes the raw JSON response into the following relational structures.

B. Data Structures and Relational Schema

The analytical pipeline maintains three primary relational structures to ensure data lineage from ingestion to inference.

- **Table I: `WEATHER_HISTORICAL`**
  Acts as the *Source Layer*, storing 60 days of multi-dimensional historical records (Temperature, Precipitation, Geolocation) for both Cupertino and San Jose. It serves as the ground-truth training set for the ML model.
- **Table II: `WEATHER_FORECAST`**
  Acts as the *Inference Layer*, capturing the 7-day predictive output from Snowflake Cortex. It includes point estimates (`TEMP_MAX`) and 95% confidence intervals (`LOWER/UPPER_BOUND`), with identifiers manually projected to resolve series nullification.
- **Table III: `WEATHER_FINAL_REPORT`**
  Acts as the *Report Layer*, created via a `UNION ALL` operation. It consolidates 120 historical records and 14

forecasted records into a unified 67-day time-series, optimized for direct visualization in Snowflake Dashboards.

TABLE I: Schema for `WEATHER_HISTORICAL` (Source Layer)

Column	Type	Description
LOCATION_NAME	VARCHAR	City identifier (e.g., San Jose, Cupertino).
LATITUDE	FLOAT	Geographic latitude coordinate.
LONGITUDE	FLOAT	Geographic longitude coordinate.
DATE	DATE	Observation date (YYYY-MM-DD).
TEMP_MAX	FLOAT	Maximum daily temperature (Celsius).
TEMP_MIN	FLOAT	Minimum daily temperature (Celsius).
TEMP_MEAN	FLOAT	Mean daily temperature (Celsius).
PRECIPITATION	FLOAT	Daily total precipitation (mm).
WEATHER_CODE	INTEGER	WMO weather interpretation code.

TABLE II: Schema for `WEATHER_FORECAST` (Inference Layer)

Column	Type	Description
LOCATION_NAME	VARCHAR	Geographic identifier (Cupertino, San_Jose) retrieved via series projection.
DATE	DATE	The 7-day future prediction horizon (2026-03-03 to 2026-03-09).
TEMP_MAX	FLOAT	Predicted maximum temperature (Point Estimate).
LOWER_BOUND	FLOAT	Lower threshold of the 95% confidence interval.
UPPER_BOUND	FLOAT	Upper threshold of the 95% confidence interval.
DATA_TYPE	VARCHAR	Constant label set to 'FORECAST' for downstream union operations.

TABLE III: Schema for `WEATHER_FINAL_REPORT` (Table III)

Column	Type	Description
LOCATION_NAME	VARCHAR	Unified city identifier.
DATE	DATE	67-day temporal axis.
TEMP_MAX	FLOAT	Observed or predicted temperature.
DATA_TYPE	VARCHAR	Discriminator ('ACTUAL' vs 'FORECAST').

C. Data Volume and Distribution

Upon the successful execution of the ML pipeline, the record distribution within the `WEATHER_FINAL_REPORT` was validated. The data integrity check (Table IV) confirms the expected 67-day window per city.

TABLE IV: Consolidated Record Count per Location

Location	Actual Rows	Forecast Rows	Total
San Jose	60	7	67
Cupertino	60	7	67
Total	**120**	**14**	**134**

VI. Implementation Details

The system architecture adopts a modular design, decoupling data engineering from analytical modeling through a dual-DAG orchestration strategy and a transactional ML pipeline within Snowflake. This part aligns with the **Solution Requirement** from the requirement file.For code details, please go to GitHub.

A. Dual-DAG Pipeline Orchestration

The orchestration layer is managed by **Apache Airflow**, which executes two distinct Directed Acyclic Graphs (DAGs) to maintain a clean separation between data ingestion and predictive modeling:

1) **Lab1_ETL DAG (Upstream):** Scheduled to run daily at **02:30 UTC**. This pipeline handles raw meteorological data ingestion from the Open-Meteo API, including data cleaning and schema-on-write loading.

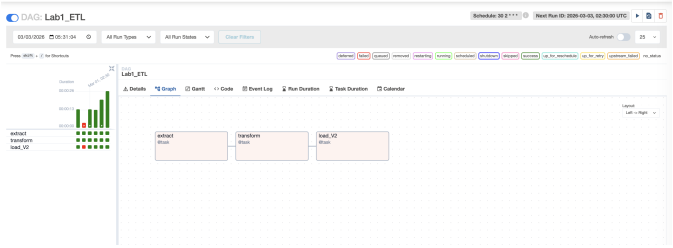

Fig. 2: Airflow Task Pipeline: The internal task sequence and operational flow of the Lab1_ETL DAG.

2) **Lab1_ML_Forecasting DAG (Downstream):** Scheduled at **02:45 UTC**. This 15-minute offset ensures the upstream ETL process has successfully committed all records to the data warehouse before the ML training phase begins.

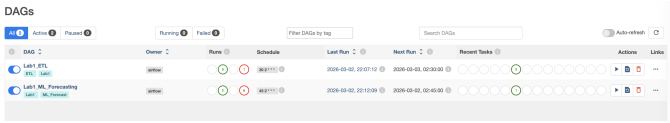

Fig. 3: Airflow DAG Management: Status overview showing the 15-minute scheduled offset between ETL and ML workflows.

B. Security and Environment Configuration

To maintain security and portability, the pipeline utilizes Airflow's built-in metadata management for sensitive credentials and environment-specific parameters:

- **Snowflake Connection:** An encrypted connection (`snowflake_conn`) manages warehouse, role, and account-level authentication, abstracted from the Python source code.

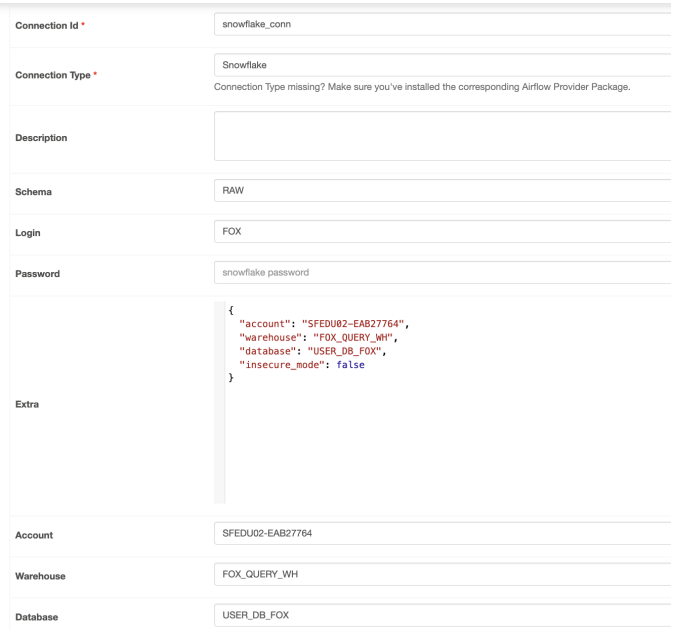

Fig. 4: Airflow Connection Configuration: Secure storage of Snowflake credentials and metadata.

- **Airflow Variables:** Dynamic identifiers (e.g., `weather_target_table`) are stored as key-value pairs. This allows for rapid table re-targeting and model naming without requiring code modifications.

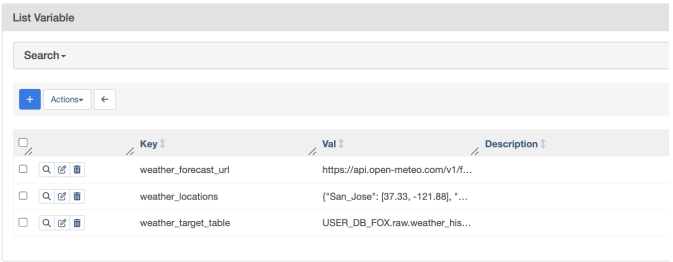

Fig. 5: Airflow Variables: Centralized management of database schema and table identifiers.

C. Snowflake Cortex ML Integration and Data Results

The core predictive logic leverages the `SNOWFLAKE.ML.FORECAST` engine. The execution results are materialized across three distinct relational layers to ensure data lineage and integrity:

1) **Table 1: Source Layer (`WEATHER_HISTORICAL`)**
Stores the 60-day ground-truth historical observations used for model training.

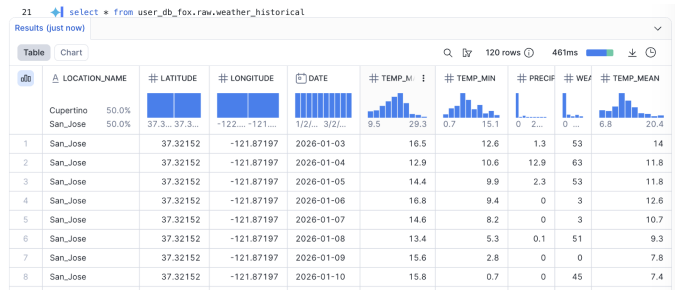

Fig. 6: Historical Data Results: The 60-day training set retrieved from the source table.

2) **Table 2: Inference Layer (`WEATHER_FORECAST`)**
Materializes the raw predictive output from the Cortex engine, including confidence intervals for both Cupertino and San_Jose.

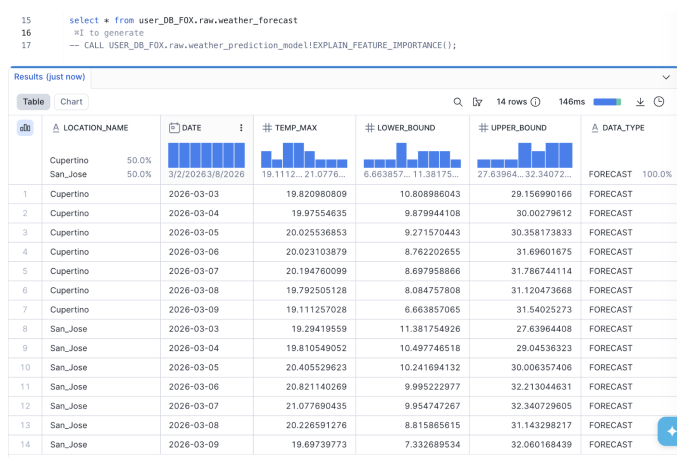

Fig. 7: Inference Layer Results: The 7-day multi-series forecast generated by Snowflake Cortex.

3) **Table 3: Consolidated Layer (`WEATHER_FINAL_REPORT`)**
The final analytical view formed by the `UNION ALL` of actual and forecasted data points.

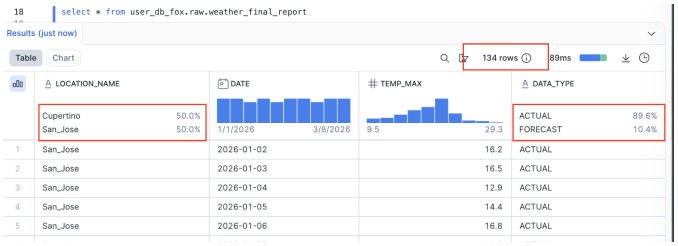

Fig. 8: Consolidated Report: The unified 134-row dataset ready for visualization.

D. Transactional Integrity and Implementation Logic

To ensure the robustness of the forecasting pipeline, the ML task is encapsulated within an explicit **SQL Transaction** framework. This architectural choice guarantees *Atomicity*: the final reporting layer is only updated if the model training, inference capture, and data union stages all succeed.

As implemented in the `ML.py` script, a `try...except` block manages the connection lifecycle. If a runtime error occurs (e.g., API timeout or Snowflake resource contention), a `ROLLBACK` command is issued to prevent partial data corruption, ensuring the database remains in a consistent state.

1) Key Technical Implementation Highlights: The following SQL procedural block (Listing 1) demonstrates the advanced techniques used to manage the ML lifecycle:

- **Dynamic Inference Capturing:** Utilizing `LET x := SQLID` and `RESULT_SCAN(:x)` to materialize transient model outputs into the physical `WEATHER_FORECAST` entity table.
- **Schema Normalization:** Explicitly `CASTING` the `SERIES` and `TS` columns to ensure data type alignment during the `UNION ALL` operation.
- **Unified CTE Integration:** Employing a **Common Table Expression (CTE)** to filter 60 days of ground-truth data before merging it with the newly generated 7-day forecast.

Listing 1: Procedural SQL for Multi-Series Inference and Consolidation

```
BEGIN
    -- Step 1: Execute Multi-Series Forecast
        (95% Confidence Interval)
    CALL weather_prediction_model!FORECAST(
        FORECASTING_PERIODS => 7,
        CONFIG_OBJECT => {'prediction_interval
            ': 0.95}
    );

    -- Step 2: Capture Query ID for Transient
        Result Materialization
    LET x := SQLID;

    -- Step 3: Materialize Table 2 (Inference
        Layer) with Type Alignment
    CREATE OR REPLACE TABLE weather_forecast
        AS
    SELECT
        CAST(SERIES AS VARCHAR) AS
            LOCATION_NAME,
        CAST(TS AS DATE) AS DATE,
        FORECAST AS TEMP_MAX,
        LOWER_BOUND, UPPER_BOUND,
        'FORECAST' AS DATA_TYPE
    FROM TABLE(RESULT_SCAN(:x));

    -- Step 4: Generate Table 3 (Consolidated
        Reporting Layer)
    CREATE OR REPLACE TABLE
        weather_final_report AS
    WITH historical_data AS (
        SELECT LOCATION_NAME, DATE, TEMP_MAX,
            'ACTUAL' AS DATA_TYPE
        FROM weather_historical
        WHERE DATE >= DATEADD('day', -60,
            CURRENT_DATE())
    )
    SELECT LOCATION_NAME, DATE, TEMP_MAX,
        DATA_TYPE FROM historical_data
    UNION ALL
    SELECT LOCATION_NAME, DATE, TEMP_MAX,
        DATA_TYPE FROM weather_forecast;
END;
```

E. System Limitations and Scope

Despite the successful deployment of the dual-DAG orchestration, certain technical limitations remain within the current implementation:

- **Univariate Modeling Constraints:** The forecasting engine currently utilizes a univariate time-series approach. Although the `TEMP_MEAN` variable was successfully integrated into the data pipeline, it is not yet utilized as an exogenous factor to refine the `TEMP_MAX` predictions.
- **Temporal Granularity:** The system is designed for daily batch processing. Due to the update frequency of the Open-Meteo API and the 15-minute offset between DAGs, the pipeline cannot support real-time or intra-day meteorological volatility monitoring.
- **Model Recalibration Overhead:** The current strategy retrains the model on every run. For larger datasets involving hundreds of cities, a more efficient "warm-start" or periodic retraining strategy would be required to optimize Snowflake credit consumption.

VII. Lessons Learned

The development and deployment of the automated weather forecasting pipeline provided several critical technical insights, ranging from environment configuration to complex SQL troubleshooting.

A. Configuration and Dependency Management

A significant portion of the initial development time was spent on troubleshooting connection failures between *Apache Airflow* and *Snowflake*.

- **Credential Integrity:** An initial oversight involved neglecting to update the `Snowflake_conn` and *Airflow Variables* in the local environment. This led to persistent authentication errors and null value exceptions. We learned that in containerized environments, ensuring the

synchronization between the *Docker* metadata and the *Airflow* metadata is a foundational prerequisite that must be validated before executing any DAG logic.

- **Variable Parameterization:** Using hardcoded paths initially hindered portability. Transitioning to *Airflow Variables* taught us the importance of decoupling orchestration logic from environmental configuration.

B. SQL Compilation and Schema Alignment

Technical challenges within the Snowflake environment underscored the complexity of in-database machine learning:

- **Strict Signature Matching:** We encountered a `STATEMENT_ERROR` related to the `SNOWFLAKE.ML.FORECAST` function. The compiler failed to resolve the subquery signature until we implemented the `TABLE()` constructor. This taught us that cloud-native ML functions often require explicit type casting and formal dataset encapsulation to be correctly interpreted by the SQL engine.
- **Schema Drift in ML Outputs:** The machine-generated headers from Cortex (`SERIES`, `TS`) did not align with our historical table. We learned that explicit aliasing is mandatory to prevent `UNION ALL` failures and to maintain the integrity of the unified reporting layer.

C. Transactional Integrity and Version Control

- **Atomicity via Transactions:** Early iterations produced fragmented data when tasks failed midway. By implementing explicit `BEGIN/COMMIT` blocks, we ensured the system maintains *ACID* properties, guaranteeing that the final report only updates upon total pipeline success.
- **Git Hygiene:** Through the synchronization process, we realized the importance of a robust `.gitignore` strategy. Preventing local logs and `__pycache__` from polluting the repository is essential for professional collaborative development and clear audit trails.

VIII. Future Work

To further enhance the scalability and predictive accuracy of the system, three primary areas for future development have been identified based on the current implementation's performance.

A. Architectural Shift: Transitioning from ETL to ELT

Future iterations will focus on migrating the data processing logic from a traditional ETL pattern to a cloud-native **ELT (Extract, Load, Transform)** framework. By implementing *Snowpipe* for automated, continuous data ingestion and *Snowflake Dynamic Tables* for incremental transformations, the system will:

- Reduce the computational overhead on the Airflow orchestration layer.
- Enable near real-time data availability for the ML training set.
- Simplify the pipeline maintenance by utilizing SQL-based declarative transformations.

B. Predictive Variance Diagnosis: Addressing "Mean-Reverting" Bias

As illustrated in Fig.9, the current model demonstrates limited variance in the 7-day forecast, resulting in a nearly flat temperature gradient. This phenomenon, often referred to as "mean-reversion," indicates that the model is prioritizing long-term historical averages over short-term stochastic fluctuations.

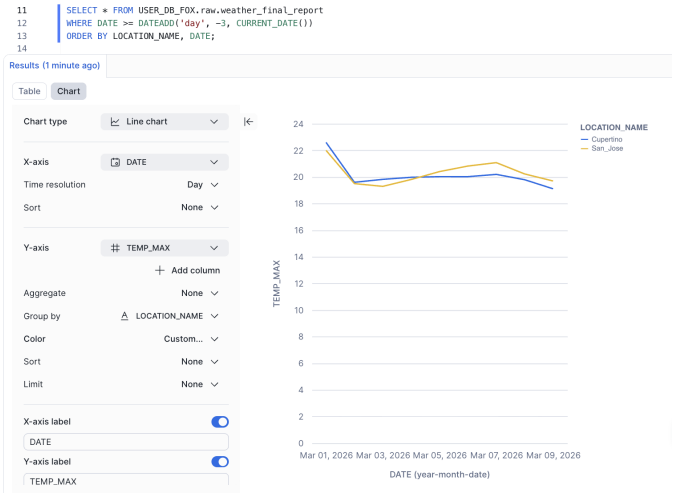

Fig. 9: Analysis of Forecast Variance: Observed lack of daily fluctuations in predicted `TEMP_MAX`

C. Multi-Variate Optimization: Feature Engineering Expansion

To overcome the current variance limitations, future development will transition from a univariate approach to **Multi-Variate Feature Engineering**. We plan to refine the model's feature vector by incorporating exogenous variables that drive meteorological volatility:

- **Cyclical Time Encodings:** Applying Sine and Cosine transformations to the "Day of Year" variable to help the model better recognize annual seasonality patterns.
- **Atmospheric Covariates:** Integrating humidity, barometric pressure, and cloud cover as predictors to improve sensitivity to sudden weather fronts.
- **Advanced Ensembles:** Evaluating the impact of combining *Snowflake Cortex* with external exogenous datasets to account for regional geographic influences.

IX. Conclusion

This project successfully demonstrates the design and implementation of an automated, cloud-native weather forecasting system. By integrating **Apache Airflow** for robust task orchestration and **Snowflake Cortex** for high-performance in-database machine learning, we developed a pipeline that transforms raw API data into actionable predictive insights with minimal human intervention.

Through the iterative debugging process, we addressed critical challenges related to environment synchronization, SQL function signatures, and transactional data integrity. The final

architecture not only provides a continuous 67-day time-series visualization for multiple geographic locations but also establishes a scalable template for future ELT and advanced data science initiatives. This experiment underscores the efficacy of modern data platforms in managing the complexities of real-time predictive analytics.

REFERENCES

[1] github/Lab1, "Weather Forecasting Pipeline with Snowflake Cortex and Apache Airflow," GitHub Repository, 2026. [Online]. Available: https://github.com/xuwang-goldenwater/sjsu-data226/tree/main/Lab1.